

Design and Analysis of Algorithms

Lecture 5: Divide and Conquer Algorithms (Part 1)



Excerpts from the previous lectures

- The analysis of the complexity of a recurrence relation involves finding the asymptotic upper bound on the running time of a recursive algorithm. This is usually done by finding a closed-form expression for the number of operations performed by the algorithm as a function of the input size and then determining the order of growth of the expression as the input size becomes large.
- Solving recurrences plays a crucial role in the analysis, design, and optimization of algorithms, and is an important topic in computer science. There are mainly three ways of solving recurrences:
 - Substitution Method
 - Master Theorem Method
 - Tree Method





Table of Contents for this topic

- Divide and Conquer Approach
- Searching: binary search
- Finding maximum and minimum
- Powering a number
- Sorting: merge sort and quicksort
- Matrix multiplication: Strassen's algorithm





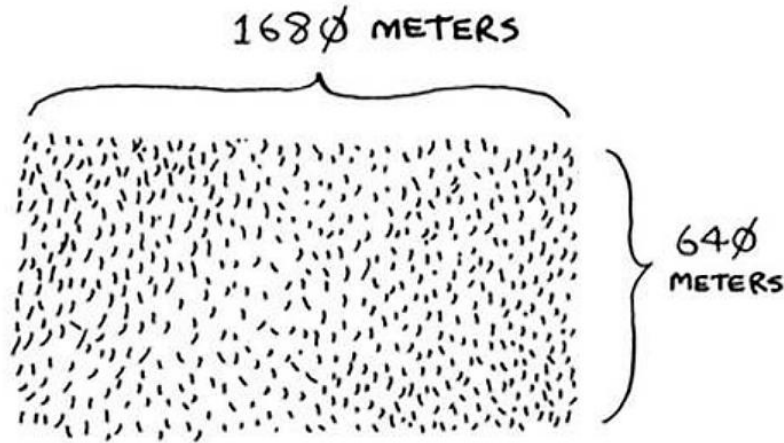
Agenda for this lecture

- Divide and Conquer Approach
- Searching: binary search
- Finding maximum and minimum
- Powering a number



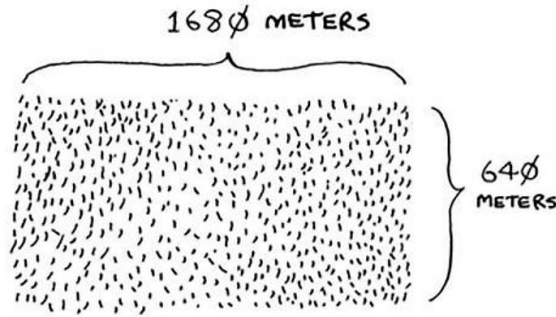
Divide and Conquer

- D&C can take some time to grasp, so let's start with an example.
- Suppose you're a farmer with a plot of land shown below. You want to divide this farm evenly into square plots.

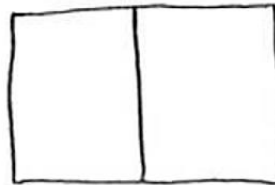


Divide and Conquer

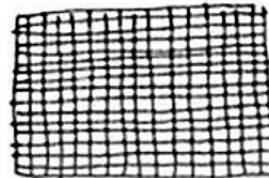
- As you want to divide this farm evenly into square plots.



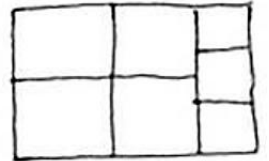
- So, none of these will work. →



BOXES ARE
NOT SQUARE



BOXES ARE TOO
SMALL



ALL BOXES MUST
BE SAME SIZE

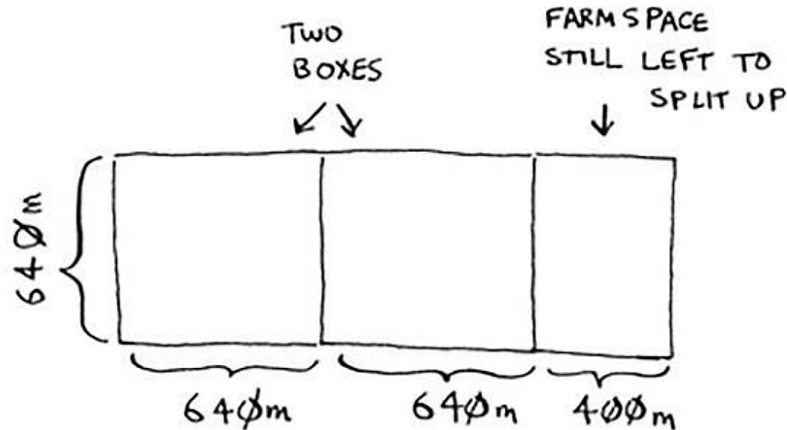
Divide and Conquer

- How do you figure out the largest square size you can use for a plot of land?
- Use the Divide and Conquer (D&C strategy)! D&C algorithms are recursive algorithms.
- To solve a problem using D&C, there are two steps:
 - ✓ **Figure out the base case. This should be the simplest possible case.**
 - ✓ **Divide or decrease your problem until it becomes the base case.**



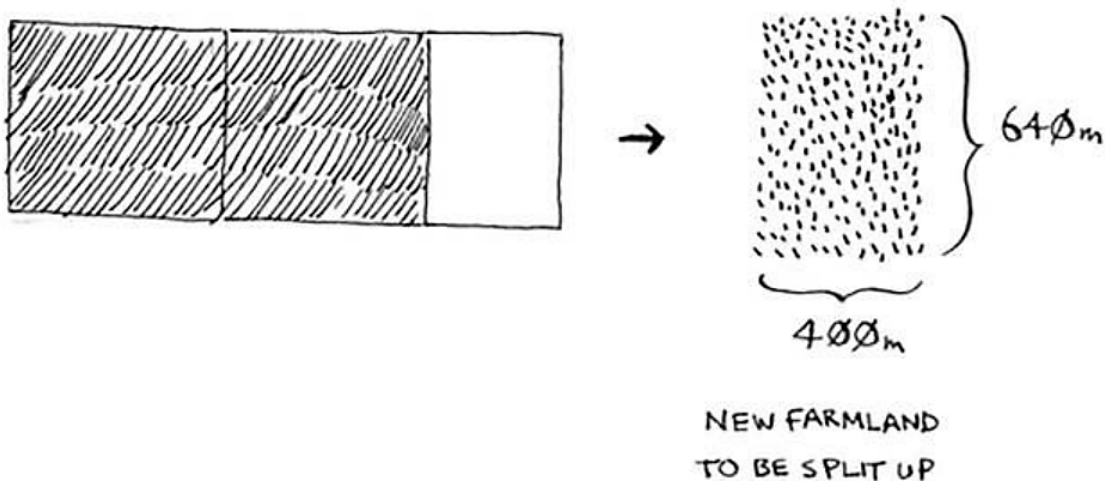
Divide and Conquer

- Let's use D&C to find the solution to this problem. What is the largest square size you can use?
- First, figure out the base case. The easiest case would be if one side was a multiple of the other side.



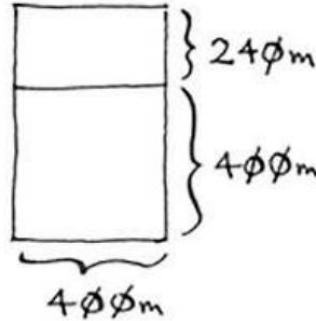
Divide and Conquer

- You can fit two 640×640 boxes in there, and there's some land still left to be divided. Now here comes the "Aha!" moment. Here's a farm segment left to divide. Why don't you apply the same algorithm to this segment?

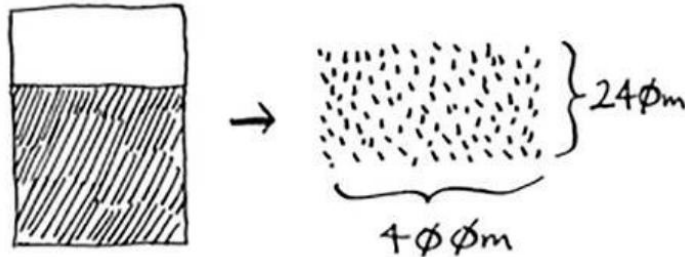


Divide and Conquer

- Let's apply the same algorithm again. Starting with a $640 \times 400\text{m}$ farm, the biggest box you can create is $400 \times 400\text{ m}$.

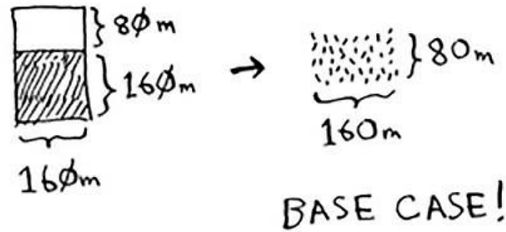


- And that leaves you with a smaller segment, $400 \times 240\text{ m}$. →

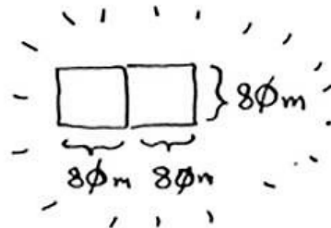


Divide and Conquer

- And then you draw a box on that to get an even smaller segment.

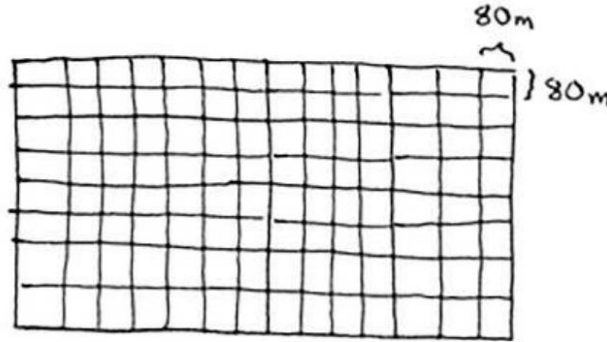


- Hey, you're at the base case: 80 is a factor of 160. If you split up this segment using boxes, you don't have anything left over!



Divide and Conquer

- So, for the original farm, the biggest plot size you can use is 80×80 m.



- To recap, here's how D&C works: figure out a simple case as the base case, figure out how to reduce your problem and get to the base case.
- D&C isn't a simple algorithm that you can apply to a problem. Instead, it's a way to think about a problem.



Divide and Conquer

- Let's do one more example. You're given an array of numbers, as shown below.



- You have to add up all the numbers and return the total. It's pretty easy to do this with a loop:

```
def sum(arr):  
    total = 0  
    for x in arr:  
        total += x  
    return total
```

```
print sum([1, 2, 3, 4])
```

← Calling the function with
your input array



Divide and Conquer

- But how would you do this with a recursive function?
- Step 1: Figure out the base case. What's the simplest array you could get? Think about the simplest case and then read on. If you get an array with 0 or 1 element, that's pretty easy to sum up.

BASE CASE { $[]$ 0 ELEMENTS = SUM IS 0
 $[7]$ 1 ELEMENT = SUM IS 7



Divide and Conquer

- Step 2: You need to move closer to an empty array with every recursive call. How do you reduce your problem size? Here's one way.

$$\text{sum}(\boxed{2 \mid 4 \mid 6}) = 12$$

It's the same as this.

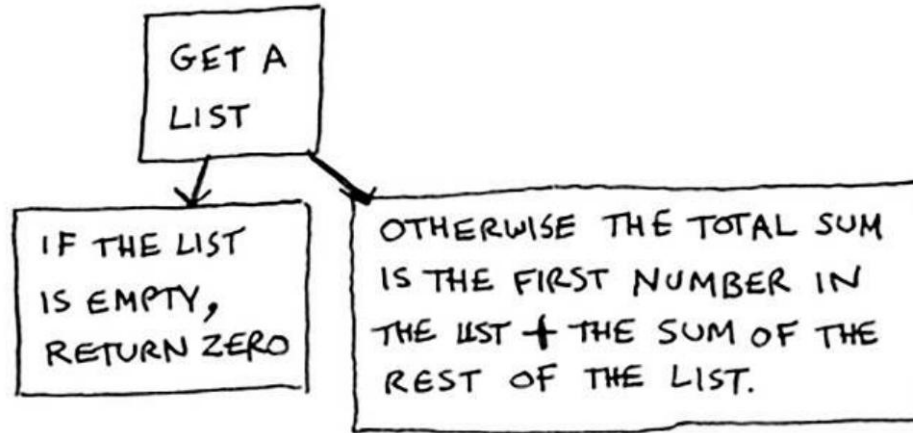
$$2 + \text{sum}(\boxed{4 \mid 6}) = 2 + 10 = 12$$

- In either case, the result is 12. But in the second version, you're passing a smaller array into the sum function. That is, you decreased the size of your problem!



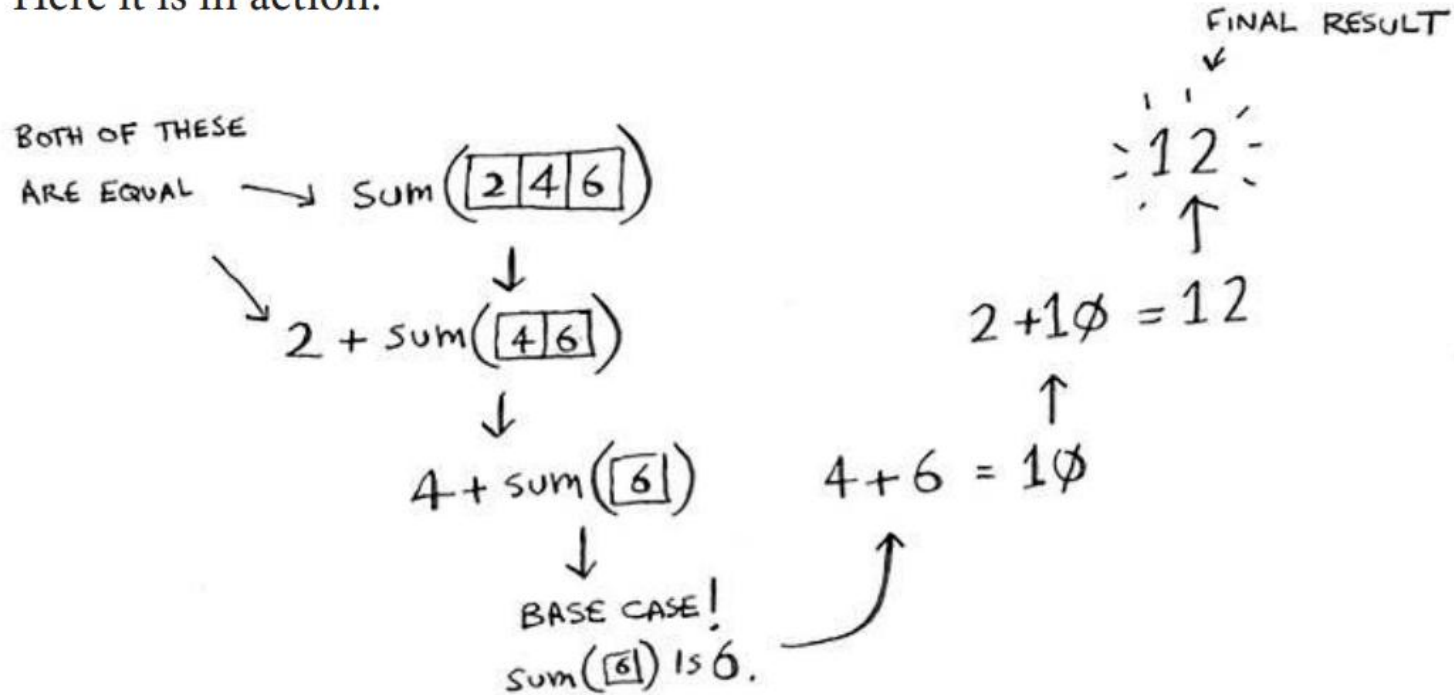
Divide and Conquer

- Your sum function could work like this.



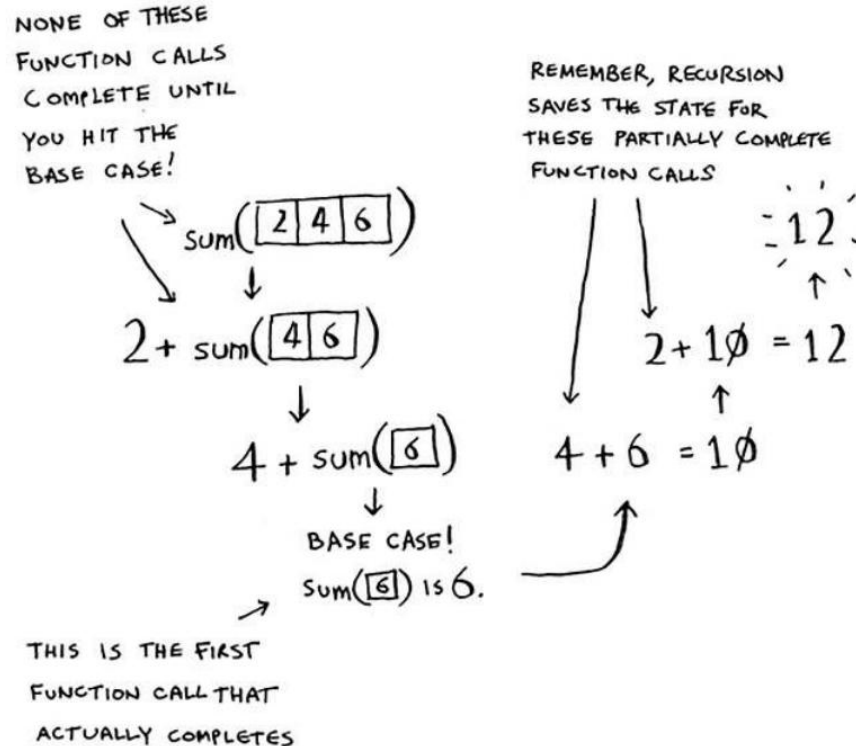
Divide and Conquer

Here it is in action.



Divide and Conquer

- Remember, recursion keeps track of the state



Tip:

When you're writing a recursive function involving an array, the base case is often an empty array or an array with one element. If you're stuck, try that first.

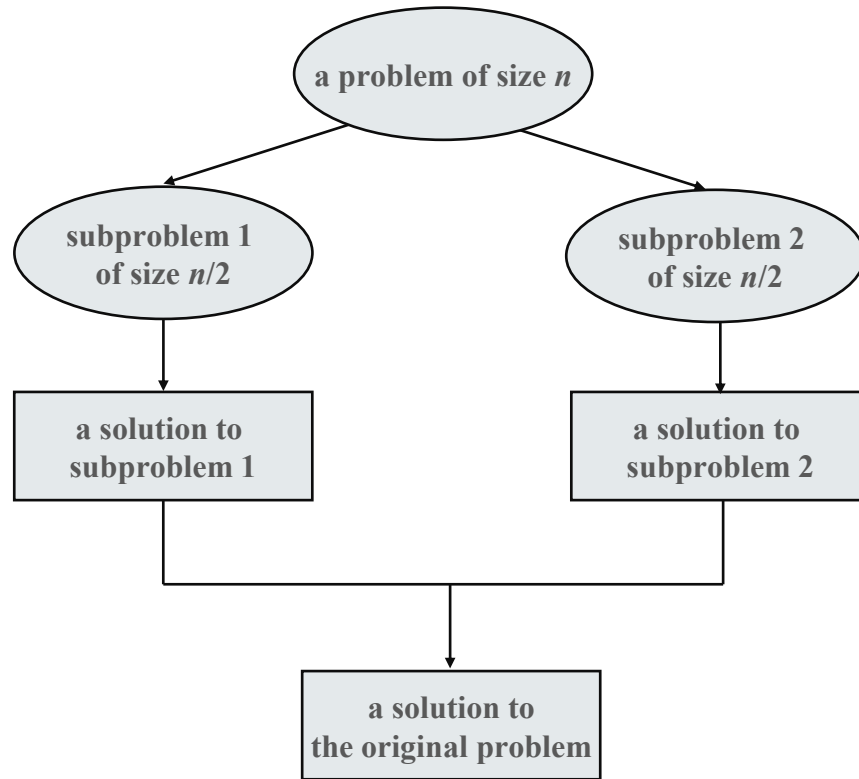


Divide and Conquer

- The most-well known algorithm design strategy is based on the following steps:
 - ✓ **Divide:** a problem to be solved is broken into a number of subproblems of the same form as the original problems.
 - ✓ **Conquer:** the subproblems are then solved independently, usually recursively.
 - ✓ **Combine:** the solutions to the subproblems are combined to provide the answer to the original problem.



Divide and Conquer



Divide and Conquer Examples

- Searching: binary search
- Finding maximum and minimum
- Powering a number
- Sorting: merge sort and quicksort
- Matrix multiplication: Strassen's algorithm



Example (1): We are given a sorted array A of n elements and a target value x . Our task is to determine the index i such that $A[i] = x$, or return a specific value (such as 0 or -1) if the target is not present in the array.

The Naïve Algorithm:
Linear Search

The Efficient Algorithm:
Binary Search



Linear search (1/2)

- **Linear Search:** is a **brute-force algorithm** that sequentially checks each element of the array, from start to finish, until the target is found or the end is reached. It makes no assumption about the order of the data, treating the array as an unstructured list. Its simplicity is its main advantage, but this comes at the cost of inefficiency on large datasets.

Searched Element

39



13	9	21	15	39	19	27
0	1	2	3	4	5	6



Linear search (2/2)

```
FUNCTION LinearSearch(A, x)
  // A: the sorted array of size n
  // x: the target value to find

  n = length(A)
  FOR i FROM 0 TO n-1 DO
    IF A[i] == x THEN
      RETURN i    // Target found at index i
    END IF
  END FOR
  RETURN -1      // Target not found
END FUNCTION
```

✓ Time Complexity: $O(n)$

- Best Case: $\Omega(1)$. The target is the first element.
- Worst Case: $O(n)$. The target is the last element or not present. Every element must be inspected.
- Average Case: $\Theta(n)$. On average, we expect to check $\sim n/2$ elements.



Binary search (1/5)

- Binary Search: is a classic divide-and-conquer algorithm. It exploits the sorted nature of the array to eliminate half of the remaining search space with each step.
- The core idea is to compare the target with the middle element; based on this single comparison, it can definitively discard one of the two halves. This leads to a dramatic reduction in the number of steps required.



Binary search (2/5)

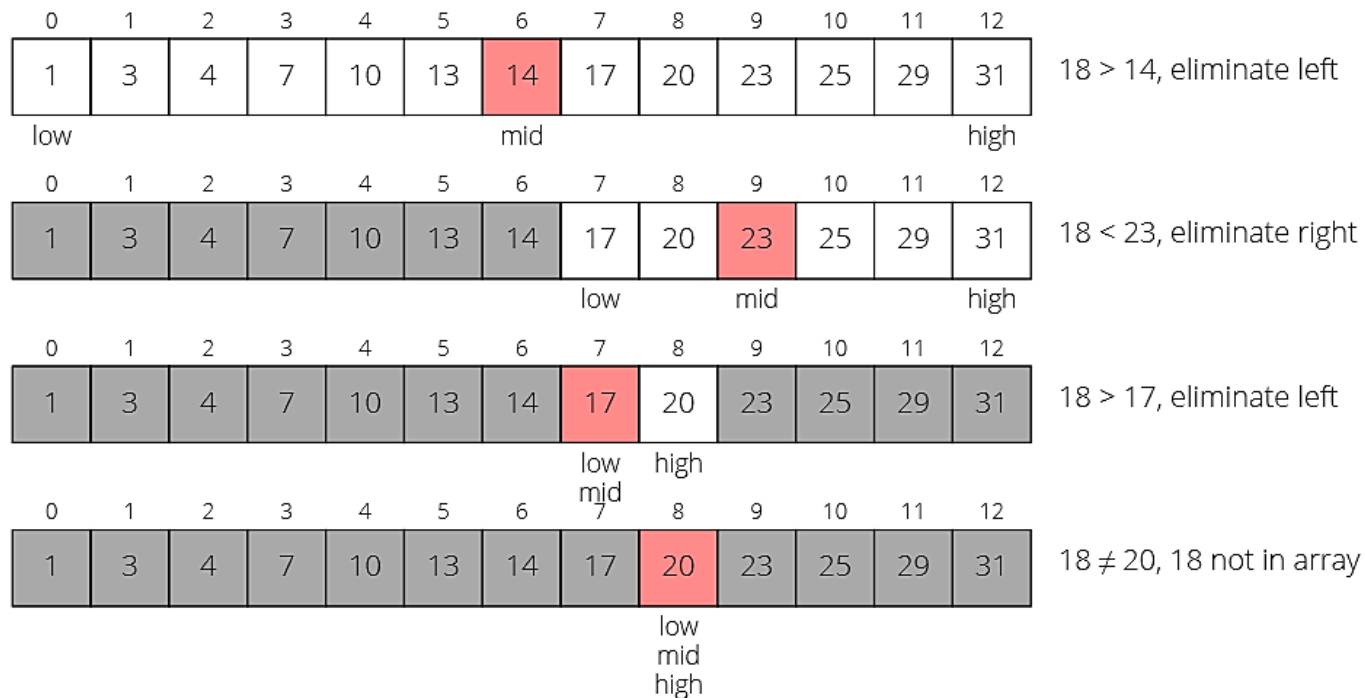
Binary Search for $x = 7$

0	1	2	3	4	5	6	7	8	9	10	11	12	
1	3	4	7	10	13	14	17	20	23	25	29	31	$7 < 14$, eliminate right
low			mid				high						
0	1	2	3	4	5	6	7	8	9	10	11	12	
1	3	4	7	10	13	14	17	20	23	25	29	31	$7 > 4$, eliminate left
low		mid		high									
0	1	2	3	4	5	6	7	8	9	10	11	12	
1	3	4	7	10	13	14	17	20	23	25	29	31	$7 < 10$, eliminate right
low			mid	high									
0	1	2	3	4	5	6	7	8	9	10	11	12	
1	3	4	7	10	13	14	17	20	23	25	29	31	7 found at index 3!
low			mid	high									



Binary search (3/5)

Binary Search for $x = 18$



Binary search (4/5)

```
FUNCTION recursiveBinarySearch(A, x, low, high)
    // Base Case 1: Search space is invalid
    IF low > high THEN
        RETURN -1
    END IF

    mid = low + (high - low) // 2

    // Base Case 2: Target found
    IF A[mid] == x THEN
        RETURN mid
    // Recursive Case 1: Search left subarray
    ELSE IF x < A[mid] THEN
        RETURN recursiveBinarySearch(A, x, low, mid - 1)
    // Recursive Case 2: Search right subarray
    ELSE
        RETURN recursiveBinarySearch(A, x, mid + 1, high)
    END IF
END FUNCTION
```

Recurrence Relation:

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

● Time Complexity: $O(\log n)$ by master method

- Best Case: $\Omega(1)$. The target is the first element.
- Worst Case: $O(\log n)$. The target is the last element or not present. Every element must be inspected.
- Average Case: $\Theta(\log n)$. On average, we expect to check $\sim n/2$ elements.



Binary search (5/5)

Search for target value **23** in the sorted array:

[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

```
FUNCTION binarySearchRecursive(A, 23, 0, 9)
|   low = 0, high = 9
|   mid = (0 + 9) // 2 = 4
|   A[mid] = A[4] = 16
|   Compare: 23 > 16 → Search RIGHT
|
└─ CALL: binarySearchRecursive(A, 23, 5, 9)
    |   low = 5, high = 9
    |   mid = (5 + 9) // 2 = 7
    |   A[mid] = A[7] = 56
    |   Compare: 23 < 56 → Search LEFT
    |
    └─ CALL: binarySearchRecursive(A, 23, 5, 6)
        |   low = 5, high = 6
        |   mid = (5 + 6) // 2 = 5
        |   A[mid] = A[5] = 23
        |   Compare: 23 == 23 → FOUND!
        |
        └─ RETURN 5 ← Success!
```



Linear search vs. Binary search

The table below shows the maximum number of comparisons needed for Binary Search and Linear Search

Array Length	Linear Search	Binary Search
10	10	4
10^2	10^2	7
10^3	10^3	10
10^4	10^4	14
10^5	10^5	17
10^6	10^6	20
10^7	10^7	24
10^8	10^8	27
10^9	10^9	30

For problem 1:
Should you choose linear
search or binary search?



Example (2): Find maximum and minimum element in an unsorted array.



Brute Force
Approach

Divide and Conquer
Approach



Brute Force Approach

- The brute force approach is straightforward: iterate through the entire array once, comparing each element to the current maximum and current minimum, updating them as needed.
- **Time Complexity: $O(n)$.**

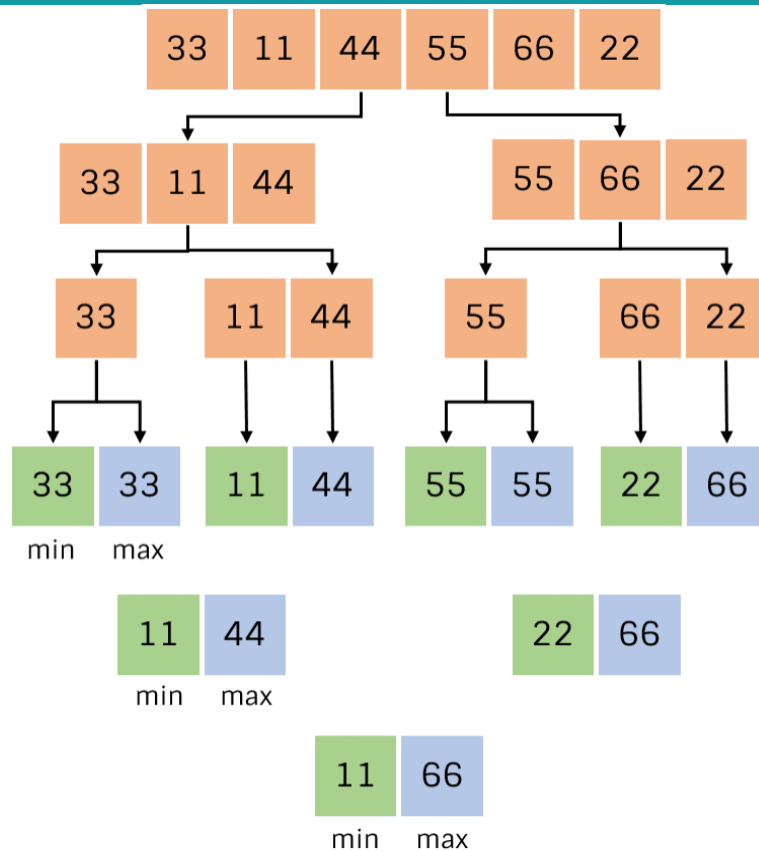
```
FUNCTION findMaxMinBruteForce(A)
  // A: input array of size n
  n = length(A)
  IF n == 0 THEN
    RETURN NULL, NULL  // Handle empty array
  END IF

  max = A[0]
  min = A[0]

  FOR i FROM 1 TO n-1 DO
    IF A[i] > max THEN
      max = A[i]  // Update max if current element is larger
    END IF
    IF A[i] < min THEN
      min = A[i]  // Update min if current element is smaller
    END IF
  END FOR

  RETURN max, min
END FUNCTION
```

Divide and Conquer Approach



Divide and Conquer Approach

- Recurrence Relation: $T(n) = 2T(\frac{n}{2}) + 1$
- According to master method, time complexity is $O(n)$.

```
// Recursive helper function
FUNCTION findMaxMinRecursive(A, low, high)
    // Base case: single element
    IF low == high THEN
        RETURN A[low], A[low]    // max = min = single element
    END IF

    // Base case: two elements
    IF high == low + 1 THEN
        IF A[low] > A[high] THEN
            RETURN A[low], A[high]
        ELSE
            RETURN A[high], A[low]
        END IF
    END IF

    // Divide: split the array into two halves
    mid = low + (high - low) // 2

    // Conquer: recursively find max/min of left and right halves
    leftMax, leftMin = findMaxMinRecursive(A, low, mid)
    rightMax, rightMin = findMaxMinRecursive(A, mid + 1, high)

    // Combine: compare results from left and right halves
    overallMax = MAX(leftMax, rightMax)
    overallMin = MIN(leftMin, rightMin)

    RETURN overallMax, overallMin
END FUNCTION
```

Divide and Conquer Approach

Complete Execution Trace: $A[0..7] = [3, 7, 2, 9, 1, 8, 5, 4]$

```
FUNCTION findMaxMinRecursive(A, 0, 7)
|   low = 0, high = 7
|   mid = (0 + 7) // 2 = 3
|
|   └─ LEFT RECURSIVE CALL: findMaxMinRecursive(A, 0, 3)
|       |   low = 0, high = 3
|       |   mid = (0 + 3) // 2 = 1
|       |
|       |   └─ LEFT-LEFT CALL: findMaxMinRecursive(A, 0, 1)
|           |   low = 0, high = 1
|           |   Base Case (high == low + 1): TRUE
|           |   Compare A[0]=3 and A[1]=7
|           |   IF A[0] > A[1]? 3 > 7? FALSE
|           |   RETURN max = A[1] = 7, min = A[0] = 3
|           |   ↳ leftMax = 7, leftMin = 3
|           |
|           |   └─ LEFT-RIGHT CALL: findMaxMinRecursive(A, 2, 3)
|               |   low = 2, high = 3
|               |   Base Case (high == low + 1): TRUE
|               |   Compare A[2]=2 and A[3]=9
|               |   IF A[2] > A[3]? 2 > 9? FALSE
|               |   RETURN max = A[3] = 9, min = A[2] = 2
|               |   ↳ rightMax = 9, rightMin = 2
|               |
|           └─ COMBINE LEFT HALF:
|               |   overallMax = MAX(leftMax, rightMax) = MAX(7, 9) = 9
|               |   overallMin = MIN(leftMin, rightMin) = MIN(3, 2) = 2
|               |
|           └─ RETURN max = 9, min = 2
|               ↳ leftMax = 9, leftMin = 2
```

Divide and Conquer Approach

Complete Execution Trace: $A[0..7] = [3, 7, 2, 9, 1, 8, 5, 4]$

```
└─ RIGHT RECURSIVE CALL: findMaxMinRecursive(A, 4, 7)
|   |   low = 4, high = 7
|   |   mid = (4 + 7) // 2 = 5
|   |
|   └─ RIGHT-LEFT CALL: findMaxMinRecursive(A, 4, 5)
|       |   low = 4, high = 5
|       |   Base Case (high == low + 1): TRUE
|       |   Compare A[4]=1 and A[5]=8
|       |   IF A[4] > A[5]? 1 > 8? FALSE
|       |   RETURN max = A[5] = 8, min = A[4] = 1
|       |   ↳ leftMax = 8, leftMin = 1
|       |
|       └─ RIGHT-RIGHT CALL: findMaxMinRecursive(A, 6, 7)
|           |   low = 6, high = 7
|           |   Base Case (high == low + 1): TRUE
|           |   Compare A[6]=5 and A[7]=4
|           |   IF A[6] > A[7]? 5 > 4? TRUE
|           |   RETURN max = A[6] = 5, min = A[7] = 4
|           |   ↳ rightMax = 5, rightMin = 4
|           |
|           └─ COMBINE RIGHT HALF:
|               |   overallMax = MAX(leftMax, rightMax) = MAX(8, 5) = 8
|               |   overallMin = MIN(leftMin, rightMin) = MIN(1, 4) = 1
|               |
|               └─ RETURN max = 8, min = 1
|                   ↳ rightMax = 8, rightMin = 1
|
|   └─ COMBINE FINAL RESULTS:
|       |   overallMax = MAX(leftMax, rightMax) = MAX(9, 8) = 9
|       |   overallMin = MIN(leftMin, rightMin) = MIN(2, 1) = 1
|
|   └─ RETURN max = 9, min = 1
```

Brute Force Approach Vs. Divide and Conquer

- For problem 2:

Aspect	Brute Force	Divide and Conquer
Time Complexity	$O(n)$	$O(n)$
Algorithm Style	Iterative	Recursive
Code Complexity	Simple	More Complex
Practical Performance	Better (no recursion overhead)	Slightly slower



Example (3): Develop efficient algorithms to compute a^n and analyze their time.



Brute Force Approach

- The brute force approach multiplies a by itself n times using iterative multiplication.
- **Time Complexity: $O(n)$.**

```
FUNCTION powerNaive(a, n)
    // a: base number
    // n: non-negative exponent

    IF n == 0 THEN
        RETURN 1
    END IF

    result = 1
    FOR i FROM 1 TO n DO
        result = result * a
    END FOR

    RETURN result
END FUNCTION
```



Divide and Conquer Approach

- The divide and conquer approach uses the mathematical insight that:

$$\text{If } n \text{ is even: } a^n = a^{\frac{n}{2}} \cdot a^{\frac{n}{2}}$$

$$\text{If } n \text{ is odd: } a^n = a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a$$

- This reduces the problem size by half at each step.



Divide and Conquer Approach

- Recurrence Relation: $T(n) = T(\frac{n}{2}) + 1$
- According to master method, time complexity is $O(\log n)$.

```
FUNCTION powerDivideConquer(a, n)
    // a: base number
    // n: non-negative exponent

    // Base case
    IF n == 0 THEN
        RETURN 1
    END IF

    // Recursive case
    half = powerDivideConquer(a, n // 2) // Integer division

    IF n % 2 == 0 THEN
        // n is even
        RETURN half * half
    ELSE
        // n is odd
        RETURN half * half * a
    END IF
END FUNCTION
```

Divide and Conquer Approach

Example Execution

Compute 2^8 :

```
power(2, 8)
  → power(2, 4)
    → power(2, 2)
      → power(2, 1)
        → power(2, 0) = 1
          RETURN 1 * 1 * 2 = 2
        RETURN 2 * 2 = 4
      RETURN 4 * 4 = 16
    RETURN 16 * 16 = 256
```



Divide and Conquer Approach

Comparison Table: Powering a Number – Brute Force vs Divide & Conquer

Aspect	Naive Algorithm	Divide and Conquer
Time Complexity	$O(n)$	$O(\log n)$
Algorithm Style	Iterative	Recursive
Memory Usage	Minimal	Recursion stack overhead (recursive version)
Real-world Usage	Rarely used in production	Standard in all programming languages and math libraries



Any Question?

